

LAPORAN PRAKTIKUM 2
PEMOGRAMAN BEORIENTASI OBJEK
RELASI ANTAR OBJEK (CLASS DIAGRAM & ASOSIASI)



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu:
Nurfiah, S.St, M.Kom

Asisten Praktikum:
Muhammad Dawi Syauqi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

A. Langkah-Langkah

1. Buat Class Diagram Transaksi

a. Buat Class dengan nama Transaksi

```
public class Transaksi_2511532024 {
```

Baris ini digunakan untuk membuat sebuah class bernama *Transaksi_2511532024*. Kata *public* berarti class ini dapat diakses oleh class lain.

b. Tambahkan State/Atribut dan Constructor

```
String idTransaksi;
```

Baris ini membuat atribut *idTransaksi* dengan tipe data *String*. Atribut tersebut digunakan untuk menyimpan identitas atau kode dari setiap transaksi.

```
String jenis;
```

Baris ini membuat atribut *jenis* dengan tipe data *String*. Atribut ini digunakan untuk menyimpan jenis transaksi.

```
double nominal;
```

Baris ini membuat atribut *nominal* dengan tipe data *double*. Atribut tersebut digunakan untuk menyimpan jumlah uang dalam transaksi.

```
public Transaksi_2511532024(String id, String jenis, double nominal) {
```

Baris ini membuat *constructor* untuk class *Transaksi_2511532024*. *Constructor* digunakan untuk memberikan nilai awal pada atribut ketika objek *Transaksi_2511532024* dibuat. Parameter *id* digunakan untuk menerima *ID* transaksi, *jenis* menerima jenis transaksi, sedangkan *nominal* menerima jumlah uang transaksi.

```
this.idTransaksi = id;
```

Baris ini memasukkan nilai dari parameter *id* ke atribut *idTransaksi*. *this.idTransaksi* menunjukkan atribut yang berada di dalam objek, sedangkan *id* merupakan

parameter yang diterima oleh <i>constructor</i> .

<i>this.jenis = jenis;</i>

Baris ini memasukkan nilai dari parameter <i>jenis</i> ke atribut <i>jenis</i> . Penggunaan <i>this.jenis</i> menunjukkan bahwa <i>jenis</i> yang berada di sebelah kiri adalah atribut milik objek, sedangkan <i>jenis</i> di sebelah kanan merupakan parameter <i>constructor</i> .

<i>this.nominal = nominal;</i>

Baris ini memasukkan nilai dari parameter <i>nominal</i> ke atribut <i>nominal</i> . Dengan demikian, jumlah transaksi yang diberikan ketika objek dibuat akan disimpan di dalam objek tersebut.
--

c. Buat Method

<i>public void cetakDetail() {</i>

Baris ini membuat method bernama <i>cetakDetail()</i> . Method ini digunakan untuk menampilkan informasi dari sebuah objek transaksi. Kata <i>void</i> menunjukkan bahwa method ini tidak mengembalikan nilai.
--

<i>System.out.println("ID: " + idTransaksi + " Jenis: " + jenis + " Nominal: Rp" + nominal);</i>
--

Baris ini digunakan untuk menampilkan detail transaksi ke layar. <i>System.out.println()</i> berfungsi mencetak teks, sedangkan tanda + digunakan untuk menggabungkan teks dengan nilai dari atribut <i>idTransaksi</i> , <i>jenis</i> , dan <i>nominal</i> .

2. Membuat Relasi pada Rekening.java

<i>ArrayList<Transaksi_2511532024> riwayatTransaksi;</i>
--

Baris ini membuat atribut bernama <i>riwayatTransaksi</i> dengan tipe <i>ArrayList<Transaksi_2511532024></i> . Artinya, <i>riwayatTransaksi</i> digunakan untuk menyimpan banyak objek <i>Transaksi_2511532024</i> dalam satu rekening.

```
public Rekening_2511532024(String nomor, String nama, double saldoAwal) {
```

Baris ini merupakan *constructor* dari *class Rekening_2511532024*. *Constructor* ini dijalankan ketika objek rekening baru dibuat. Parameter nomor digunakan untuk nomor rekening, nama untuk nama pemilik, dan saldoAwal untuk saldo awal.

```
    nomorRekening = nomor;
```

Baris ini memasukkan nilai parameter nomor ke atribut *nomorRekening*, sehingga nomor rekening yang diberikan akan disimpan dalam objek rekening.

```
    namaPemilik = nama;
```

Baris ini memasukkan nilai parameter nama ke atribut *namaPemilik*, sehingga nama pemilik rekening tersimpan dalam objek.

```
    saldo = saldoAwal;
```

Baris ini memasukkan nilai parameter *saldoAwal* ke atribut *saldo*. Dengan demikian, rekening memiliki saldo awal sesuai nilai yang dimasukkan.

```
    this.riwayatTransaksi = new ArrayList<>();
```

Baris ini membuat *ArrayList* baru untuk menyimpan riwayat transaksi. *new ArrayList<>()* membuat daftar yang awalnya kosong. Nantinya objek *Transaksi_2511532024* yang dibuat saat transaksi berhasil akan disimpan di dalam daftar tersebut.

```
    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
```

Baris ini menampilkan pesan bahwa rekening berhasil dibuat. Nilai *namaPemilik* digabungkan dengan teks sehingga nama pemilik ikut ditampilkan.

3. Mencatat Riwayat Transaksi di Method *setorTunai*

```
public void setorTunai(double nominal) {
```

Baris ini membuat *method setorTunai()* yang digunakan untuk melakukan

penyetoran uang ke rekening. Parameter *nominal* bertipe *double* digunakan untuk menerima jumlah uang yang ingin disetor. *void* berarti method ini tidak mengembalikan nilai.

```
if (nominal > 0) {
```

Baris ini mengecek apakah nominal yang ingin disetor lebih dari 0. Jika nilainya lebih dari 0, penyetoran dianggap valid dan program akan menjalankan kode di dalam *if*.

```
saldo += nominal;
```

Baris ini menambahkan jumlah uang yang disetor ke saldo rekening. Operator *+=* berarti nilai nominal ditambahkan ke nilai saldo yang sudah ada.

```
String idTrx = "TRX-S-" + System.currentTimeMillis();
```

Baris ini membuat variabel *idTrx* bertipe *String* untuk menyimpan *ID* transaksi. Teks "TRX-S-" digunakan sebagai awalan *ID* untuk transaksi setor, kemudian digabungkan dengan *System.currentTimeMillis()* yang menghasilkan waktu saat ini dalam bentuk angka. Hasilnya digunakan sebagai *ID* transaksi yang berbeda berdasarkan waktu pembuatannya.

```
Transaksi_2511532024 trxBaru = new Transaksi_2511532024(idTrx, "Kredit",  
nominal);
```

Baris ini membuat objek baru bernama *trxBaru* dari class *Transaksi_2511532024*. *Constructor* menerima tiga nilai, yaitu *idTrx* sebagai *ID* transaksi, "Kredit" sebagai jenis transaksi karena transaksi tersebut merupakan penyetoran, dan *nominal* sebagai jumlah uang yang disetor.

```
riwayatTransaksi.add(trxBaru);
```

Baris ini memasukkan objek *trxBaru* ke dalam *ArrayList* *riwayatTransaksi*. Dengan demikian, transaksi setor yang berhasil akan tersimpan sebagai riwayat transaksi

rekening.

```
System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
```

Baris ini menampilkan pesan bahwa penyetoran berhasil sekaligus menampilkan nominal yang disetor dan saldo rekening setelah penyetoran.

```
} else {
```

Baris ini menunjukkan bagian yang dijalankan apabila kondisi nominal > 0 bernilai salah. Artinya, nominal yang dimasukkan adalah 0 atau kurang dari 0.

```
System.out.println("Gagal: Nominal setor harus lebih dari 0!");
```

Baris ini menampilkan pesan bahwa penyetoran gagal karena nominal yang dimasukkan harus lebih dari 0.

B. Tugas/Latihan Praktikum

1. Integrasi Rekam Jejak Penarikan

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java
- Tambahkan logika perekaman objek transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai “Debit” dan kodenya menggunakan prefiks “TRX-T-“. Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

a. Class Rekening.

```
public void tarikTunai(double nominal) {  
    if (nominal < 10000) {  
        System.out.println("Transaksi Gagal: Minimal penarikan  
RP10.000");  
    } else if (nominal > saldo){  
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi.  
saldo anda Rp" + saldo );  
    } else {  
        saldo -= nominal;  
        String idTrx = "TRX-T-" + System.currentTimeMillis();
```

```

        Transaksi_2511532024 trxBaru = new Transaksi_2511532024(idTrx,
"Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + " berhasil.
Saldo saat ini: Rp" + saldo);
    }
}

```

Penjelasan Code:

a. Class Rekening

```
public void tarikTunai(double nominal) {
```

Baris ini merupakan deklarasi *method tarikTunai*. Kata *public* berarti method dapat dipanggil dari *class* lain. *void* berarti method ini tidak mengembalikan nilai. *tarikTunai* adalah nama method yang digunakan untuk proses penarikan uang, sedangkan *double nominal* merupakan parameter yang digunakan untuk menyimpan jumlah uang yang akan ditarik.

```
if (nominal < 10000) {
```

Baris ini menggunakan *if* untuk memeriksa apakah nilai nominal kurang dari 10000. Jika jumlah uang yang ingin ditarik kurang dari Rp10.000, maka kondisi bernilai benar dan program menjalankan perintah di dalam blok *if*.

```
System.out.println("Transaksi Gagal: Minimal penarikan RP10.000");
```

Baris ini digunakan untuk menampilkan pesan ke layar menggunakan *System.out.println()*. Pesan tersebut memberitahukan bahwa transaksi gagal karena jumlah penarikan yang diperbolehkan minimal adalah Rp10.000.

```
} else if (nominal > saldo) {
```

Baris ini menggunakan *else if* untuk melakukan pemeriksaan kedua apabila kondisi *if* sebelumnya tidak terpenuhi. Kondisi *nominal > saldo* digunakan untuk mengecek apakah jumlah uang yang ingin ditarik lebih besar daripada saldo yang tersedia. Jika *nominal* lebih besar dari *saldo*, berarti *saldo* rekening tidak mencukupi.

```
System.out.println("Transaksi Gagal: Saldo tidak mencukupi. saldo anda Rp" + saldo);
```

Baris ini menampilkan pesan bahwa transaksi gagal karena saldo tidak mencukupi. Tanda + *saldo* digunakan untuk menggabungkan nilai saldo saat ini dengan teks yang ditampilkan. Jadi, program tidak hanya memberi tahu bahwa saldo tidak cukup, tetapi juga menampilkan jumlah saldo yang tersedia.

```
} else {
```

Baris ini merupakan bagian *else* yang dijalankan apabila kedua kondisi sebelumnya tidak terpenuhi. Artinya, *nominal* penarikan sudah minimal Rp10.000 dan saldo yang dimiliki cukup untuk melakukan penarikan.

```
saldo -= nominal;
```

Baris ini digunakan untuk mengurangi saldo berdasarkan jumlah uang yang ditarik. Penulisan *saldo -= nominal* memiliki arti yang sama dengan *saldo = saldo - nominal*.

```
String idTrx = "TRX-T-" + System.currentTimeMillis();
```

Baris ini membuat *ID* untuk transaksi. *String* digunakan karena *ID* transaksi berupa teks. Variabel *idTrx* digunakan untuk menyimpan *ID* tersebut. Bagian "TRX-T-" menjadi awalan *ID*, sedangkan *System.currentTimeMillis()* menghasilkan waktu saat ini dalam satuan milidetik.

```
Transaksi_2511532024 trxBaru = new Transaksi_2511532024(idTrx, "Debit", nominal);
```

Baris ini membuat objek transaksi baru dari *class Transaksi_2511532024*. Objek tersebut disimpan dalam variabel *trxBaru*. *Constructor* menerima tiga data, yaitu *idTrx* sebagai *ID* transaksi, "Debit" sebagai jenis transaksi karena terjadi pengurangan saldo, dan *nominal* sebagai jumlah uang yang ditarik.

```
riwayatTransaksi.add(trxBaru);
```

Baris ini digunakan untuk memasukkan objek *trxBaru* ke dalam *riwayatTransaksi*. Dengan begitu, transaksi penarikan yang berhasil akan tersimpan dan nantinya dapat ditampilkan kembali ketika pengguna melihat riwayat atau mutasi rekening.

```
System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
```

Baris ini menampilkan pesan bahwa penarikan berhasil. Nilai nominal digunakan untuk menampilkan jumlah uang yang ditarik, sedangkan nilai saldo digunakan untuk menampilkan saldo setelah penarikan dilakukan.

2. Fitur Cetak Mutasi Rekening

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam mutasi tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakdetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: “Belum ada Transaksi pada rekening ini”.
- Buka file Main.java, tambahkan menu baru: “6. Cetak Mutasi (Riwayat)”. Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

a. Class Rekening

```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening ini");  
    } else {  
        System.out.println("--- MUTASI REKENING ---");  
        for (Transaksi_2511532024 transaksi : riwayatTransaksi) {  
            transaksi.cetakDetail();  
        }  
    }  
}
```

b. Class Main

```
case 6:
    akunAktif.cetakMutasi();
break;
```

Penjelasan Code:

a. Class Rekening

```
public void cetakMutasi() {
```

Baris ini merupakan deklarasi method *cetakMutasi()*. Kata *public* berarti *method* dapat dipanggil dari *class* lain. *void* berarti method tidak mengembalikan nilai. *cetakMutasi* adalah nama *method* yang digunakan untuk menampilkan riwayat atau mutasi transaksi pada rekening.

```
if (riwayatTransaksi.isEmpty()) {
```

Baris ini menggunakan *if* untuk mengecek apakah *riwayatTransaksi* masih kosong. *Method .isEmpty()* digunakan untuk mengetahui apakah list tidak memiliki data transaksi. Jika tidak ada transaksi sama sekali, kondisi ini bernilai *true*.

```
System.out.println("Belum ada transaksi pada rekening ini");
```

Jika *riwayatTransaksi* kosong, baris ini akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

```
} else {
```

Baris ini digunakan ketika kondisi *if* tidak terpenuhi. Artinya, *riwayatTransaksi* sudah memiliki setidaknya satu data transaksi, sehingga program akan menjalankan kode yang ada di dalam *else*.

```
System.out.println("--- MUTASI REKENING ---");
```

Baris ini menampilkan judul --- MUTASI REKENING --- sebagai penanda bahwa data yang akan ditampilkan berikutnya adalah daftar riwayat transaksi rekening.

<i>for (Transaksi_2511532024 transaksi : riwayatTransaksi) {</i>
--

Baris ini menggunakan <i>for-each</i> untuk mengambil setiap data transaksi yang terdapat di dalam <i>riwayatTransaksi</i> satu per satu. <i>Transaksi_2511532024</i> menunjukkan bahwa setiap data dalam list berupa objek dari <i>class</i> tersebut. Variabel transaksi digunakan untuk menyimpan satu objek transaksi yang sedang diproses. Tanda : dapat dibaca sebagai "dari", sehingga maksudnya adalah mengambil setiap transaksi dari <i>riwayatTransaksi</i> .
--

<i>transaksi.cetakDetail();</i>

Baris ini memanggil <i>method cetakDetail()</i> dari objek <i>transaksi</i> . Method tersebut digunakan untuk menampilkan detail dari transaksi yang sedang diproses. Karena berada di dalam perulangan <i>for</i> , setiap transaksi yang ada di <i>riwayatTransaksi</i> akan ditampilkan detailnya satu per satu.

b. Class Main

<i>case 6:</i>

Baris ini menunjukkan pilihan menu ke-6 dalam struktur <i>switch-case</i> yaitu cek mutasi.

<i>akunAktif.cetakMutasi();</i>

Baris ini memanggil <i>method cetakMutasi()</i> dari objek <i>akunAktif</i> . <i>akunAktif</i> merupakan rekening yang sedang digunakan oleh pengguna, sedangkan <i>cetakMutasi()</i> digunakan untuk menampilkan seluruh riwayat transaksi dari rekening tersebut.

<i>break;</i>

<i>break</i> digunakan untuk menghentikan proses pada <i>case 6</i> setelah <i>cetakMutasi()</i> selesai dijalankan. Program kemudian keluar dari struktur <i>switch</i> dan melanjutkan ke bagian kode berikutnya.

Output:

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi (Riwayat)  
0. Keluar  
Pilih Menu: 6  
|--- MUTASI REKENING ---  
ID: TRX-S-1789808758033 | Jenis: Kredit | Nominal: Rp20000.0  
ID: TRX-T-1789808763220 | Jenis: Debit | Nominal: Rp193873.0
```